Predictive Maintenance and Monitoring of Simulated Industrial Equipment on AWS

Akshay Kripalani, Joel Jacob, Kaivalya Joglekar,
Kanish Jain, Manas Kulkarni and Rama Varshney
Department of Computer Engineering
Mukesh Patel School of Technology Management and Engineering
NMIMS, Mumbai, India
Rama Varshney: Assistant Professor
akshay.kripalani74@nmims.in, joelchirayil.jacob78@nmims.in
kaivalyarakesh.joglekar57@nmims.in, kanish.jain82@nmims.in
manas.kulkarni73@nmims.in, rama.varshney@nmims.edu

Abstract. This paper presents a deployed AWS system for monitoring seven simulated industrial assets and predicting faults within the next five simulated minutes. One EC2 gateway represents three induction motors, a centrifugal pump, an air compressor, a cooling fan and a generator. Signed telemetry passes through API Gateway, Lambda and a FIFO queue into DynamoDB and an S3 archive. Configurable deterioration profiles produce thermal and mechanical faults. A dataset of 420 independent simulation runs supports local training and evaluation. The trained random forest is exported as portable learned trees and served by a separate inference Lambda. On 76 held-out runs, fault-warning precision was 90.3%, recall was 94.3%, and the median warning lead was five simulated minutes. The deployed system demonstrated warnings before faults, automatic cooldown and restart, and mechanical faults requiring operator repair. An authenticated React dashboard combines measurements, predictions, maintenance history and bounded historical reports. The results establish a reproducible teaching demonstration; they do not establish predictive accuracy on real industrial equipment.

Keywords: Predictive maintenance · Cloud computing · Industrial simulation · Random forest · Serverless inference · AWS

1 Introduction

Equipment monitoring becomes more useful when it gives an operator time to act before a fault occurs. Condition-based maintenance links observed equipment condition to maintenance decisions [1]. A college demonstration must also make the data path, failure behavior and resulting actions visible without depending on physical sensors or an expensive deployment.

The implemented platform combines a configurable simulator, retry-safe telemetry processing, learned fault warnings, two maintenance paths and a protected dashboard. Its objective is to demonstrate these functions together and support them with reproducible tests and measured evidence. The deployment uses the Sydney region and a USD 50 project budget.

The contribution is an integrated teaching implementation. Scheduled deterioration makes demonstrations repeatable; inference uses observed measurement windows rather than hidden failure times, allowing a learned warning before threshold activation.

2 Background and Design Rationale

A threshold rule identifies a condition that has already crossed a limit. Prognostic methods instead use condition information to anticipate future deterioration [1]. Random forests combine multiple learned decision trees [2] and provide a compact classifier for the small tabular feature set used here. We use scikit-learn for training [3], then export the tree structures so that cloud inference does not require a scientific Python installation.

Overlapping measurement windows are dependent. Randomly splitting them would place closely related samples in training and testing, producing an optimistic estimate. Following scikit-learn's guidance on suitable splits [4], we separate entire simulation episodes before fitting the model or choosing its warning threshold. Episodes share the simulator's assumptions, so this measures performance within that synthetic domain.

The cloud design separates ingestion, processing and inference. AWS documents that Lambda consumers of SQS can receive messages more than once [5]. Event identifiers and conditional DynamoDB transactions [6] therefore protect telemetry state under retries, while prediction failure remains separate from the durable monitoring path.

3 System Implementation

3.1 Telemetry and Cloud Architecture

Figure 1 shows the deployed architecture. One EC2 instance runs the seven assets and sends a batch every ten seconds using its IAM role and signed API requests. API Gateway invokes an ingestion Lambda, which validates registered identities, timestamps, numeric bounds and schema fields. HTTP 202 means that the readings reached the FIFO queue, not that downstream work has completed.

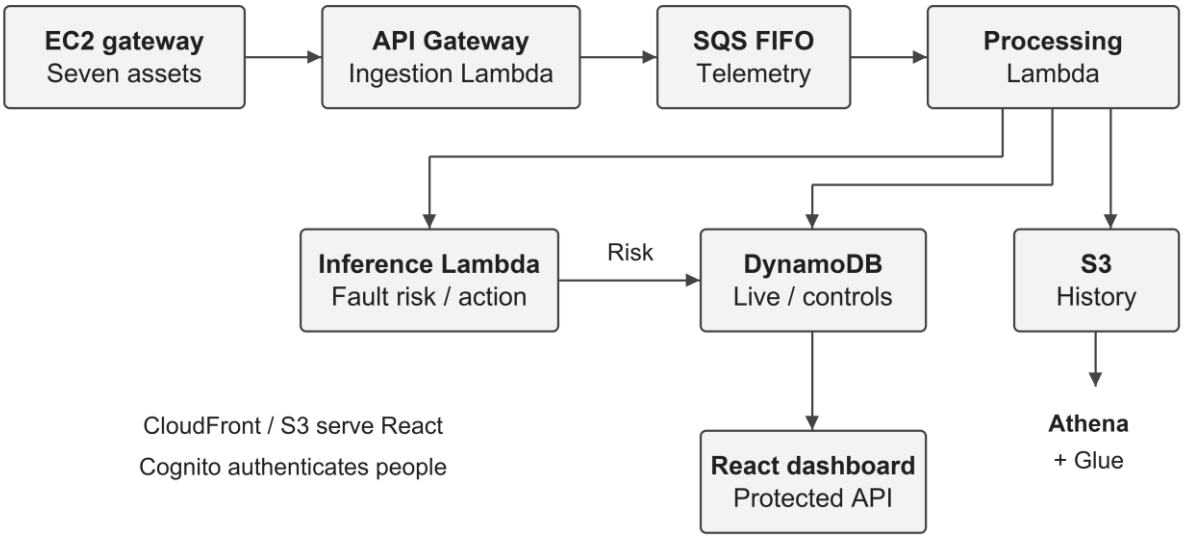


Fig. 1. Main AWS data path. Inference adds fault risk and maintenance actions to live state; S3 history is queried through Athena and Glue

The processing Lambda archives each batch as compressed, date-partitioned NDJSON in private S3. For each event, a DynamoDB transaction commits its deduplication marker, latest state, recent reading and any alert transition or notification intent. Older observations cannot replace newer state. A failed batch can be retried, and repeatedly failing messages move to a dead-letter queue. S3 and DynamoDB are not one transaction; stable archive identities and retry-safe writes handle partial completion.

Figure 2 follows a reading through monitoring and prediction. Processing asynchronously invokes the predictor after durable telemetry work. A minute schedule also reconciles inference and maintenance. Both paths read a bounded recent window from current state rather than scanning the archive. Conditional writes prevent an older prediction from replacing a newer one. A separate scheduled outbox worker sends SNS alerts; SNS acceptance is distinguished from inbox receipt.

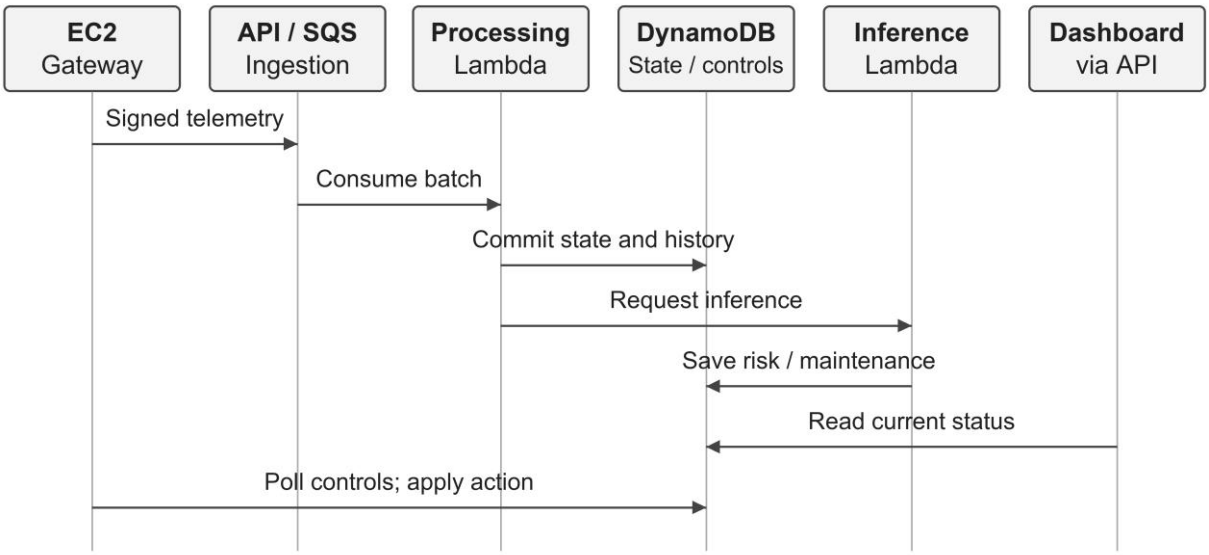


Fig. 2. Simplified telemetry, prediction and control sequence. Dashboard reads pass through the authenticated application API

3.2 Equipment and Failure Profiles

The fleet contains three induction motors, a centrifugal pump, an air compressor, a cooling fan and a generator (Table 1). Nominal temperatures, power requirements and supplementary measurements vary by type. Common temperature, vibration and power fields support comparable charts and reports. Generator output is separate from its auxiliary power consumption.

Table 1. Simulated equipment and supplementary measurements

Equipment	n	Supplementary readings
Induction motor	3	Load and shaft speed
Centrifugal pump	1	Flow and pressure
Air compressor	1	Pressure
Cooling fan	1	Airflow
Generator	1	Output power, voltage, frequency

Profiles select recoverable overheating, mechanical vibration, alternating modes or persistent overheating. A nominal 30-minute cycle can be configured between 10 and 120 simulated minutes. Seeded variation changes cycle timing by approximately ten percent; load and sensor noise vary within a run. Deterioration gradually affects measurements instead of switching directly from healthy to failed. Operators can disable deterioration or retain the manual overheat, vibration and disconnect controls.

The default demonstration uses three healthy motors whose faults are introduced manually, two automatically recoverable thermal assets (pump and fan), and two mechanical assets requiring manual repair (compressor and generator). These profiles remain configurable.

Presentation speeds of 5× and 10× advance simulated time faster while retaining ten-second transmissions. Consequently, five simulated minutes represent 30 wall-clock seconds at 10×. The dashboard displays the speed so that prediction lead time is not mistaken for elapsed real time. Mechanical damage latches the device into a stopped condition until an operator repairs or resets it.

3.3 Predictive Model and Reproducible Releases

The training script generates 420 seeded episodes across equipment types, varied cycle lengths and

demonstration speeds, including healthy episodes. It produced 57,032 pre-fault feature windows. Eight inputs describe current temperature and vibration, their trends per simulated minute, normalized power, load and recent measurement ranges. The window covers at most two simulated minutes. Absolute time since cycle start, profile settings and future fault times are excluded from the input vector.

A sample is labeled overheating or vibration when the first corresponding threshold crossing occurs within the next five simulated minutes; otherwise it is labeled healthy. Post-fault samples are excluded from this warning evaluation. All windows from an episode remain together: 273 episodes train the model, 71 select the score threshold and 76 provide the held-out test. The classifier contains 32 trees, each limited to depth eight, with a minimum of 12 samples per leaf and balanced class weights.

The warning score is one minus the forest's healthy-class score. Its validation-selected threshold is 0.80. The most likely fault class supplies the warning label. This is an uncalibrated classifier score, not a guaranteed failure probability. The exported inference implementation reproduces scikit-learn's float32 threshold comparisons; 500 held-out inputs passed a numerical parity check.

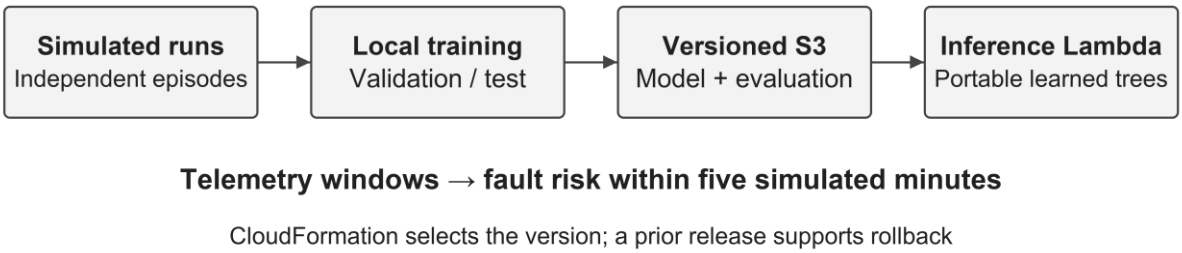


Fig. 3. Lightweight model lifecycle. Training and evaluation run locally; versioned artifacts support cloud deployment and rollback

Figure 3 shows the release process. Training dependencies are pinned in an isolated local environment. A release must meet precision and recall gates of at least 0.75 before publication. The model and evaluation are stored under a versioned S3 prefix, and CloudFormation parameters select the deployed version. Promotion and rollback use the same deployment path. Inference runs in a 256 MiB Lambda using the exported trees and standard Python, avoiding a continuously running ML server.

3.4 Maintenance and Dashboard

Three consecutive readings above 75 °C or 4.5 mm/s activate the corresponding fault; three healthy readings resolve it. Predictions provide earlier warnings while these deterministic rules remain the fault criteria. The interface uses Healthy, Warning and Fault labels, with offline connectivity shown separately. Equipment pages show type-specific readings, the five-minute outlook, model version and maintenance history. Operators can configure profiles or record repair; viewers can inspect data and reports.

The outlook displays one current forecast per device. Active faults take precedence over the model score, and stale predictions are withheld. Past warnings and actions appear separately in maintenance history, so a historical warning cannot be mistaken for a current prediction.

For recoverable overheating, the cloud controller records a cooldown request in the control table. The simulator stops the affected device, cools it toward its nominal temperature and restarts it after the bounded cooldown. A persistent thermal condition remains stopped and escalates to Repair needed; the controller does not repeatedly restart the same fault episode. Mechanical faults also require explicit operator action. Conditional control updates respect a superseding operator request. The history records predictions, cooldown, automatic restart and manual repair without adding a separate human verification workflow.

Cognito provides hosted authorization-code login with PKCE. API Gateway protects application routes,

and the backend checks operator membership before accepting controls or profile changes. Service roles supply cloud credentials. S3 buckets remain private, and CloudFront serves the React build. Recent readings are retained for 24 hours and archived telemetry for 30 days. Athena exposes three bounded reports for temperature, vibration breaches and running power, with Glue defining the archive schema. CloudWatch logs and alarms monitor platform health.

4 Evaluation and Results

4.1 Held-Out Prediction Results

Table 2 compares the forest with a causal baseline that extrapolates the current temperature and vibration trends to their fault thresholds. All reported warning metrics concern the next-five-minute binary decision on the held-out windows. The baseline uses the same window information. No test-set result was used to select the deployed model or score threshold.

Table 2. Warning performance on 76 held-out simulation episodes

Method	Precision	Recall	F1
Random forest	0.903	0.943	0.923
Linear trend baseline	0.981	0.814	0.890

The forest improves warning recall and F1 at the cost of more false positives. Its false-positive rate on healthy test windows was 0.93%, compared with 0.14% for the baseline. Of the 76 test episodes, 51 reached a fault; each received at least one warning within the preceding five simulated minutes. The median lead was five minutes and the minimum was 1.67 minutes. The remaining episodes were healthy. These figures distinguish per-window errors from episode-level detection and do not imply that every individual warning was correct.

4.2 Deployed Demonstrations and Verification

The live acceptance exercised a pump thermal episode and an air-compressor mechanical episode through the deployed API, EC2 simulator, queue, processing and inference functions. In both cases, a stored model warning preceded the active fault. The pump cooled and restarted automatically. The compressor stayed stopped with Repair needed until the authenticated operator issued a repair action. The maintenance history retained all five action categories, and a viewer's direct profile-change request was rejected. A separate live persistent-overheating test confirmed that the device remained stopped, escalated once and required manual repair. Model promotion and rollback were also verified through new live predictions. All seven assets reported online, including generator-specific output measurements.

Table 3. Verification evidence for the implementation

Layer	Result
Backend and infrastructure	86 local tests; schema and security rules passed
React browser checks	40 controlled scenarios; deployed browser verification
Model export	500 held-out predictions matched training-library output
Live maintenance	14 automatic recoveries across seven assets; explicit mechanical repair

A further controller exercise ran two accelerated overheating cycles on every asset through real EC2 telemetry and cloud inference. All seven cooled and restarted twice without manual repair, producing 14 automatic recoveries. The three/two/two default profiles were restored afterward. Regression checks also prevent pre-repair telemetry from reasserting an old fault after an acknowledged repair.

Five real browser journeys verified hosted PKCE login, live charts and mobile viewer access, automatic and manual maintenance controls, all three historical reports with CSV export, persistent team-role changes, and generator measurements with model details. Local failure-injection tests cover duplicate and older-event handling, invalid messages, storage retries and fault transitions. These checks support the defined application behavior; they are not a throughput benchmark or a guarantee of future cloud availability.

4.3 Cost and Operating Scope

The implementation retains one small EC2 simulator and adds no second host, GPU, managed ML endpoint, NAT gateway or load balancer. Local training has no AWS training charge. Inference is event-driven, with a minute reconciliation schedule and bounded retries. Its cost depends on measured execution time and the existing database writes; the seven-device expansion increases telemetry storage and processing. Initial CloudWatch observations of 84 inference calls averaged 700 ms, with a maximum of 2,586 ms including cold and release transitions. A conservative 730-hour planning model estimates USD 1.08 for inference Lambda and USD 45.47 for the complete seven-device platform before credits. The estimate includes bounded database activity and variable-service allowances, and is not a measured bill. The project tracks a USD 50 budget, with notifications at USD 20, 30 and 40. Billing snapshots are delayed and account-wide, so they must not be presented as a precise project-only cost.

Bounded history, seven-day logs, on-demand database capacity and limited historical queries constrain ongoing usage. Budget notifications provide oversight rather than a hard spending cap or automatic shutdown.

5 Limitations and Conclusion

The principal limitation is synthetic validity. Training and test episodes come from the same simulator family, and the implemented fault dynamics are simplified teaching models. No real sensor dataset, independent physical failure process or industrial safety assessment was used. The model does not estimate remaining useful life or justify autonomous repair of real equipment. Restart-based recovery applies only to the explicitly modeled transient thermal scenario; mechanical repair resets simulated damage rather than repairing hardware.

A single gateway and FIFO message group remain shared dependencies. Dashboard updates use polling, notification delivery can repeat after a worker crash, and recovery evidence covers a bounded archive exercise rather than regional disaster recovery. A useful next evaluation would introduce independently designed fault dynamics and real operating traces, measure drift and calibration, and benchmark latency and cost over a longer controlled workload.

The project demonstrates an operational predictive-maintenance loop: simulated deterioration produces telemetry, a locally trained and versioned model warns before a fault, and the cloud coordinates automatic or manual maintenance. The authenticated dashboard presents measurements, current outlooks, maintenance history and reports. Held-out evaluation and live evidence support this reproducible college-scale cloud and ML operations demonstration.

References

1. Jardine, A.K.S., Lin, D., Banjevic, D.: A review on machinery diagnostics and prognostics implementing condition-based maintenance. Mechanical Systems and Signal Processing 20(7), 1483–1510 (2006). https://doi.org/10.1016/j.ymssp.2005.09.012

2. Breiman, L.: Random forests. Machine Learning 45, 5–32 (2001). https://doi.org/10.1023/A:1010933404324

3. Scikit-learn Developers: RandomForestClassifier documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html. Accessed 27 Oct 2026

4. Scikit-learn Developers: Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html. Accessed 27 Oct 2026

5. Amazon Web Services: Using Lambda with Amazon SQS. https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html. Accessed 27 Oct 2026

6. Amazon Web Services: Amazon DynamoDB transactions: how it works. https://docs.aws.amazon.com/amazon-dynamodb/latest/developerguide/transaction-apis.html. Accessed 27 Oct 2026